

```
#!/usr/bin/env bash
#
=====
=====
====
# /home/azureuser/AGY-
MASTER/DRAFTS/05-verification-
tests.sh
#
=====
=====
====
# Forensic Post-Hardening
Verification Test Suite
# Target: Azure VM 'FatehAli'
# Status: DRAFT TEST SUITE - RUN ONLY
AFTER HARDENING POLICY IS APPLIED
#
# CHANGE LOG (fixed from previous
draft):
# - Test 1: switched from matching
sudo's error TEXT (locale/version-
dependent,
#   can silently false-pass if the
message wording differs) to checking
sudo's
```

```
# EXIT CODE, which is stable across
locales and sudo versions.
# - Test 3: the original check (`sudo
-l | grep NOEXEC`) only confirms the
tag
# is textually present in the
sudoers listing – it does not confirm
NOEXEC
# is actually enforced at runtime.
Added a real behavioral test that
# attempts a pager/shell escape
through a whitelisted command.
# - Test 5: the original rename test
doesn't distinguish "blocked by
regular
# ownership/permission" from
"blocked by chattr +i immutability" –
a
# false PASS is possible even if +i
was never applied, simply because
# normal Linux permissions already
stop non-owners. Added a second sub-
test
# that specifically checks the
immutable attribute via lsattr,
independent
# of the mv attempt's
success/failure.
# - All tests now print the exact
```

```
command and raw result for manual
audit,
#   not just PASS/FAIL, so a false
result can be caught by inspection.
#
=====
=====
=====

set -uo pipefail

PASS_COUNT=0
FAIL_COUNT=0
TOTAL_TESTS=7

echo
"=====
=====
=====
echo "POST-HARDENING VERIFICATION
TEST SUITE"
echo
"=====
=====
=====
echo "Target Date: $(date -u)"
echo "Current User: $(whoami) (UID:
$(id -u))"
echo "-----
```

```

-----
-----"

# -----
-----
-----
# TEST 1: SUDO CHATTR -I DENIAL TEST
(exit-code based, not message-text
based)
# -----
-----
-----
echo "[TEST 1] Testing 'sudo -n
chattr -i <protected file>' ..."
TARGET="/home/azureuser/AGY-
MASTER/AGENTS.md"
sudo -n chattr -i "$TARGET"
>/tmp/test1.out 2>&1
TEST1_EXIT=$?
echo "  Command exit code:
$TEST1_EXIT"
echo "  Output: $(cat
/tmp/test1.out)"
if [[ $TEST1_EXIT -ne 0 ]]; then
    echo "  RESULT: PASS (non-zero
exit – chattr was NOT permitted)"
    ((PASS_COUNT++))
else
    echo "  RESULT: FAIL (exit 0 –

```

```

chattr succeeded! sudoers whitelist
is not blocking it)"
    ((FAIL_COUNT++))
fi

# -----
-----
-----
# TEST 2: BROAD SUDO ESCALATION TEST
(whoami -> root)
# -----
-----
-----
echo "[TEST 2] Testing 'sudo -n
whoami' broad escalation ..."
WHOAMI_OUT=$(sudo -n whoami 2>&1)
TEST2_EXIT=$?
echo "  Command exit code:
$TEST2_EXIT, output: $WHOAMI_OUT"
if [[ $TEST2_EXIT -ne 0 ]]; then
    echo "  RESULT: PASS (broad
passwordless root is blocked)"
    ((PASS_COUNT++))
elif [[ "$WHOAMI_OUT" == "root" ]];
then
    echo "  RESULT: FAIL (sudo -n
whoami still returns root!
NOPASSWD:ALL is still active
somewhere)"

```

```

        ((FAIL_COUNT++))
    else
        echo "    RESULT: UNKNOWN – inspect
manually"
    fi

# -----
-----
-----
# TEST 3: NOEXEC ENFORCEMENT –
BEHAVIORAL TEST (not just config-text
check)
# -----
-----
-----
echo "[TEST 3] Testing NOEXEC is
actually enforced (pager/shell escape
attempt) ..."
# Attempt to use apt's pager
mechanism (or a direct forced shell-
escape style
# call) to spawn a subshell. If
NOEXEC is enforced, the child process
spawn
# fails or is neutered; if not, this
would hand back a shell.
# We use `apt-get -o
Debug::NoLocking=true update` piped
through a fake pager

```

```

# as a low-risk behavioral probe:
NOEXEC_PROBE=$(sudo -n env
PAGER='/bin/sh -c "echo
NOEXEC_BYPASSED_$$"' apt list --
installed 2>&1 | grep -c
"NOEXEC_BYPASSED_$$" || true)
if [[ "$NOEXEC_PROBE" -eq 0 ]]; then
    echo "  RESULT: PASS (no evidence
of subshell spawning through pager
path)"
    ((PASS_COUNT++))
else
    echo "  RESULT: FAIL (subshell
marker appeared – NOEXEC may not be
enforced for this path)"
    ((FAIL_COUNT++))
fi
echo "  NOTE: this is a best-effort
behavioral probe, not exhaustive.
Also confirm"
echo "  manually with 'sudo -l' that
NOEXEC appears against
APT_CMNDS/SYSTEMCTL_CMNDS."

# -----
-----
-----

# TEST 4: DIRECT SYSCALL UNLINK
(SYS_unlink) ON +i PROTECTED FILE

```

```

# -----
-----
-----
echo "[TEST 4] Testing direct x86_64
SYS_unlink on immutable file ..."
TEST_FILE="/home/azureuser/AGY-
MASTER/POLICIES/protected_registry.js
on"
SYSCALL_RESULT=$(python3 -c "
import ctypes, os
libc = ctypes.CDLL(None,
use_errno=True)
ret = libc.syscall(87,
'$TEST_FILE'.encode('utf-8')) #
SYS_unlink on x86_64
errno = ctypes.get_errno()
print(f'ret:{ret},errno:{errno}')
" 2>&1)
echo " Syscall result:
$SYSCALL_RESULT"
if echo "$SYSCALL_RESULT" | grep -q
"errno:1"; then
    echo " RESULT: PASS (kernel
returned EPERM – immutability held)"
    ((PASS_COUNT++))
else
    echo " RESULT: FAIL
($SYSCALL_RESULT – file may have been
deleted, check immediately)"

```



```

        ((FAIL_COUNT++))
    fi

    # -----
    -----
    # TEST 5A: PARENT DIRECTORY RENAME
    ATTEMPT (behavioral)
    # -----
    -----

    echo "[TEST 5A] Testing 'mv AGY-
    MASTER AGY-RENAME-TEST' as current
    user ..."
    RENAME_ERR=$(mv /home/azureuser/AGY-
    MASTER /home/azureuser/AGY-RENAME-
    TEST 2>&1)
    RENAME_EXIT=$?
    echo "  Exit code: $RENAME_EXIT,
    output: $RENAME_ERR"
    if [[ $RENAME_EXIT -ne 0 ]]; then
        echo "  RESULT: PASS (rename
        blocked)"
        ((PASS_COUNT++))
    else
        echo "  RESULT: FAIL (rename
        succeeded! rolling back immediately)"
        mv /home/azureuser/AGY-RENAME-
        TEST /home/azureuser/AGY-MASTER

```

```

2>/dev/null || true
    ((FAIL_COUNT++))
fi

# -----
-----
# TEST 5B: CONFIRM THE *REASON* IS
IMMUTABILITY, NOT JUST PLAIN
PERMISSIONS
# -----
-----
# Test 5A alone is not conclusive – a
plain permission denial (e.g. running
as
# a different non-owning user) would
also produce a non-zero exit here,
giving
# a false PASS even if chatr +i was
never applied. This test checks the
# immutable attribute directly via
lsattr, independent of the mv
attempt.
echo "[TEST 5B] Confirming immutable
attribute is actually set (lsattr)
..."
LSATTR_OUT=$(lsattr -d
/home/azureuser/AGY-MASTER 2>&1)

```

```

echo "  lsattr output: $LSATTR_OUT"
if echo "$LSATTR_OUT" | grep -qE '^--
--i'; then
    echo "  RESULT: PASS (immutable
'i' flag confirmed present)"
    ((PASS_COUNT++))
else
    echo "  RESULT: FAIL (immutable
flag NOT set – Test 5A may have
passed for the wrong reason, e.g.
plain permission denial rather than
immutability)"
    ((FAIL_COUNT++))
fi

# -----
-----
-----
# TEST 6: SANITY CHECK –
SUBDIRECTORIES STILL WRITABLE AFTER
LOCKING PARENT
# -----
-----
-----
echo "[TEST 6] Confirming pre-created
subdirectories remain writable inside
themselves ..."
SANITY_FILE="/home/azureuser/AGY-
MASTER/TASKS/queue/.verify-test-$$"

```

```

if touch "$SANITY_FILE" 2>/dev/null;
then
    echo "  RESULT: PASS (AGY can
still write inside TASKS/queue/ as
expected)"
    rm -f "$SANITY_FILE"
    ((PASS_COUNT++))
else
    echo "  RESULT: FAIL (cannot
write inside TASKS/queue/ -
subdirectory was likely not created
before locking the parent; AGY's
normal operation may now be broken)"
    ((FAIL_COUNT++))
fi

# -----
-----
-----
# TEST 7: AZURE IMDS IDENTITY
VERIFICATION
# -----
-----
-----
echo "[TEST 7] Testing Azure IMDS
Managed Identity token endpoint ..."
IMDS_HTTP_CODE=$(curl -s -o /dev/null
-w "%{http_code}" -H Metadata:true
"http://169.254.169.254/metadata/iden

```

```
tity/oauth2/token?api-version=2018-02-01&resource=https://management.azure.com/" 2>&1)
echo " HTTP code: $IMDS_HTTP_CODE"
if [[ "$IMDS_HTTP_CODE" == "400" ]];
then
    echo " RESULT: PASS (HTTP 400 Identity not found – zero Azure credentials on VM, unchanged)"
    ((PASS_COUNT++))
else
    echo " RESULT: FAIL (unexpected response code – investigate whether a Managed Identity was accidentally attached)"
    ((FAIL_COUNT++))
fi

echo "-----"
echo "-----"
echo "-----"
echo "VERIFICATION SUMMARY:
$PASS_COUNT / $TOTAL_TESTS TESTS
PASSED, $FAIL_COUNT FAILED"
echo
"=====
=====
====="
```

```
echo "NOTE: A single FAIL means the  
corresponding protection is not  
confirmed –"  
echo "do not treat this VM as delete-  
proof until all 7 tests PASS, and re-  
run"  
echo "this suite after any future  
config change."
```